

5. Node Classification

2026-04-18, 12:00

Tags: GNN Deep Learning Node Classification Message Passing

5.1 Message passing and Node Classification

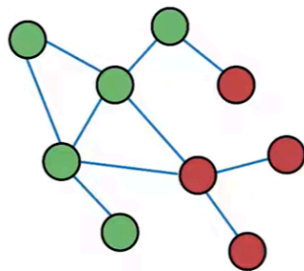
Given a network with labels on some nodes, how do we assign all other nodes in the network

In other words, semi-supervised node classification

Correlation Classification

1. Correlations Exist in Networks

- Individual behaviors are **correlated** in the network
- Correlation**: nearby nodes have the same color (belonging to the same class)

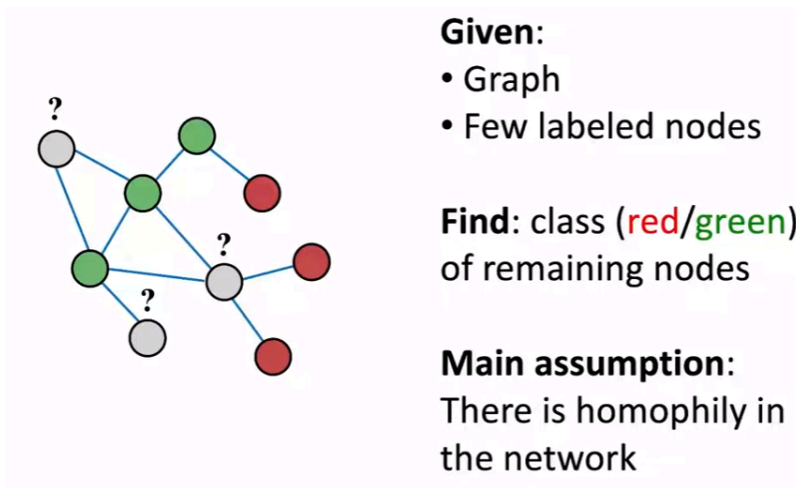


1. Homophily (Individual characteristics → social connections)
2. Influence (Social connections → Individual characteristics)
 - Homophily: The tendency of individuals to associate and bond with similar others
 - Influence: Social connections can influence the individual characteristics of a person

Principle Used: *Guilt-by-association*

If I am connected to a node with label X, then I am likely to have label X as well.

- Features of v
- Labels of the nodes in v 's neighborhood
- Features of the nodes in v 's neighborhood



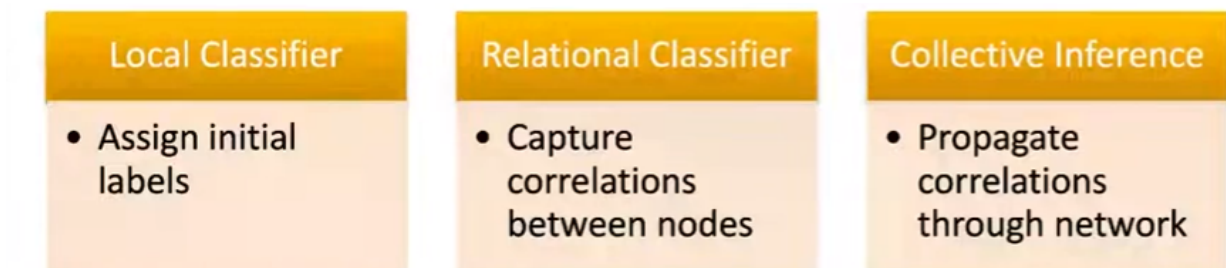
< The assumption here: there is homophily in the network >

2. Overview

1. Intuition: simultaneous classification of interlinked nodes using correlations
2. Probabilistic framework
3. Markov Assumption: the label Y_v of one node v depends on the labels of its neighbors N_v

$$P(Y_v) = P(Y_v | N_v)$$

4. Collective classification involves 3 steps:



Local Classifier

Only used for initial label assignment

- Predicts label based on node attributes/features
- Standard classification task
- *Does not use network information*

Relational Classifier

Capture correlations

- Learns a classifier to label on node based on the labels and attributes of its neighbors
- This is where network information is used

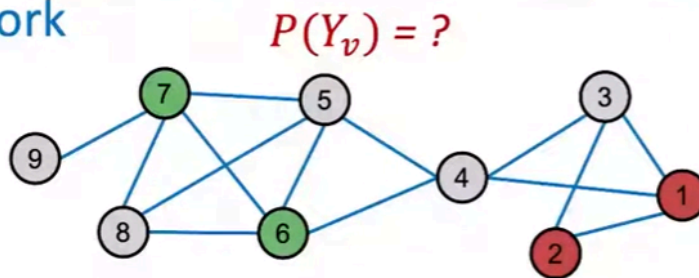
Collective Inference

Propagate the correlation

- Apply relational classifier to each node iteratively
- iterate until the inconsistency between neighboring labels is minimized
- Network structure affects the final prediction

3. Problem Setting

- How to predict the labels Y_v for the unlabeled nodes v (in grey color)?
- Each node v has a feature vector f_v
- Labels for some nodes are given (1 for green, 0 for red)
- **Task:** Find $P(Y_v)$ given all features and the network



Relational and Iterative Classification

Relational Classifier

Basic idea: Class probability Y_v of node v is a weighted average of class probabilities of its neighbors

- **Update** for each node v and label c (e.g. 0 or 1)

$$P(Y_v = c) = \frac{1}{\sum_{(v,u) \in E} A_{v,u}} \sum_{(v,u) \in E} A_{v,u} P(Y_u = c)$$

- If edges have strength/weight information, $A_{v,u}$ can be the edge weight between v and u
- $P(Y_v = c)$ is the probability of node v having label c

- **Challenges:**

- Convergence is not guaranteed
- Model cannot use node feature information

Basically, the first part is how many nodes connected with the current node; and use it

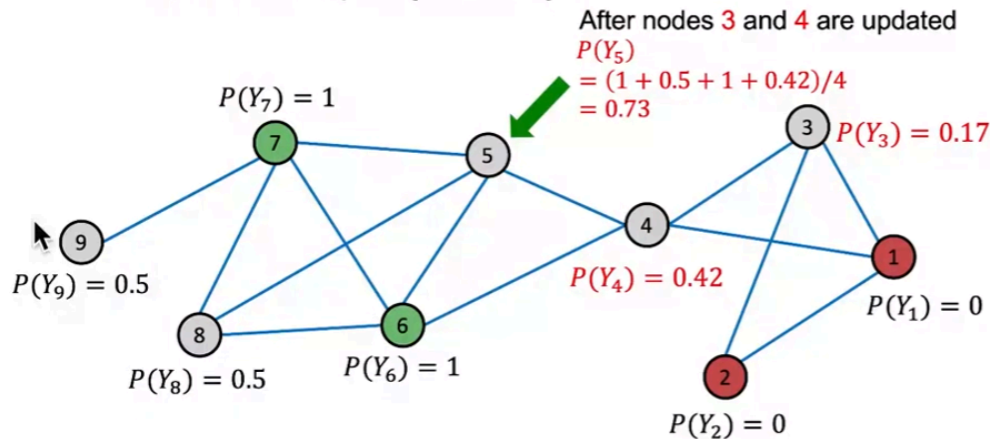
as denominator.

The second part is the sum of all the node probability = 0 or 1 sum up.

eg.

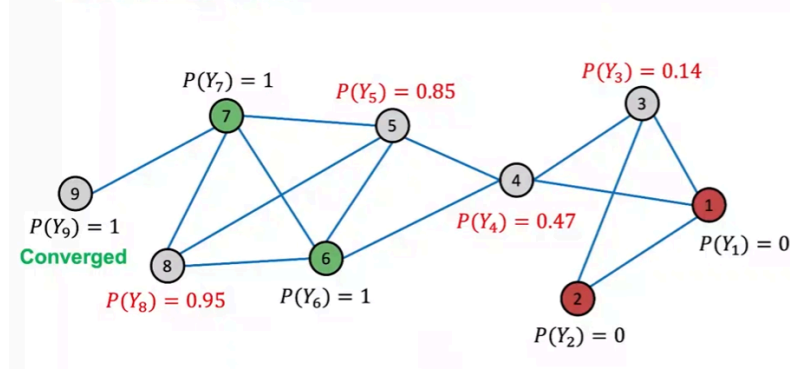
Update for the 1st Iteration:

- For node 5, $N_5 = \{4, 6, 7, 8\}$



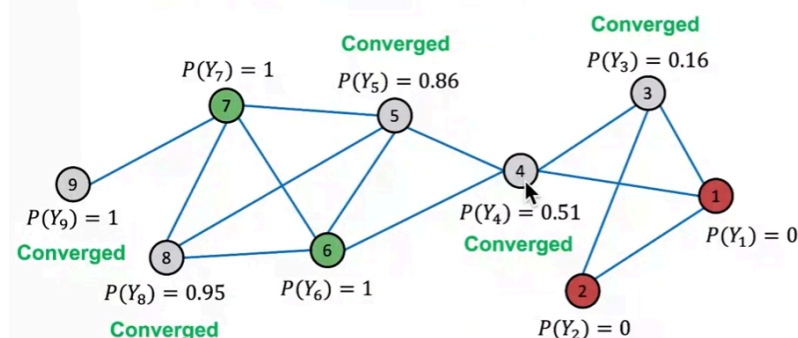
Then after multiple iterations:

After Iteration 2



And we find in the condition of convergence:

- All scores stabilize after 4 iterations. We therefore predict:
 - Nodes 4, 5, 8, 9 belong to class 1 ($P_{Y_v} > 0.5$)
 - Nodes 3 belong to class 0 ($P_{Y_v} < 0.5$)



Iterative classification

Main idea: classify node v based on its attribute f_v as well as labels Z_v of neighbor set N_v

- Specific Input type

- **Input: Graph**

- f_v : feature vector for node v
 - Some nodes v are labeled with Y_v

- **Task: Predict label of unlabeled nodes**

- **Approach: Train two classifiers:**

- $\phi_1(f_v)$ = Predict node label based on node feature vector f_v
 - $\phi_2(f_v, z_v)$ = Predict label based on node feature vector f_v and summary z_v of labels of v 's neighbors.

- Architecture

- **Phase 1: Classify based on node attributes alone**

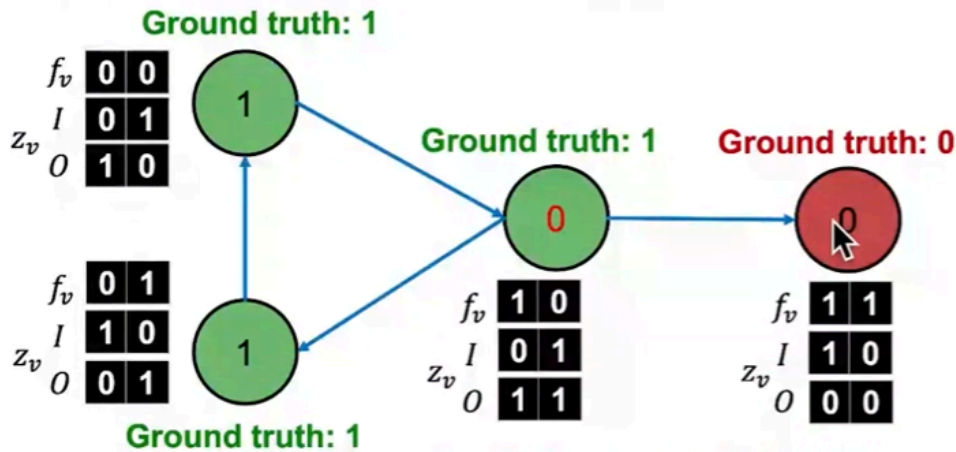
- On a **training set**, train classifier (e.g., linear classifier, neural networks, ...):
 - $\phi_1(f_v)$ to predict Y_v based on f_v
 - $\phi_2(f_v, z_v)$ to predict Y_v based on f_v and summary z_v of labels of v 's neighbors

- **Phase 2: Iterate till convergence**

- On **test set**, set labels Y_v based on the classifier ϕ_1 , compute z_v and **predict the labels with ϕ_2**
 - **Repeat** for each node v
 - Update z_v based on Y_u for all $u \in N_v$
 - Update Y_v based on the new z_v (ϕ_2)
 - Iterate until class labels stabilize or max number of iterations is reached
 - Note: Convergence is not guaranteed

- So building the summary vector Z_v is done through a list of f_v, I, O
 f_v is the feature vector of node v itself.

- Each node maintains **vectors z_v of neighborhood labels**:
 I = **Incoming** neighbor label information vector
 O = **Outgoing** neighbor label information vector
- $I_0 = 1$ if at least one of the incoming pages is labelled 0.
 Similar definitions for I_1 , O_0 , and O_1

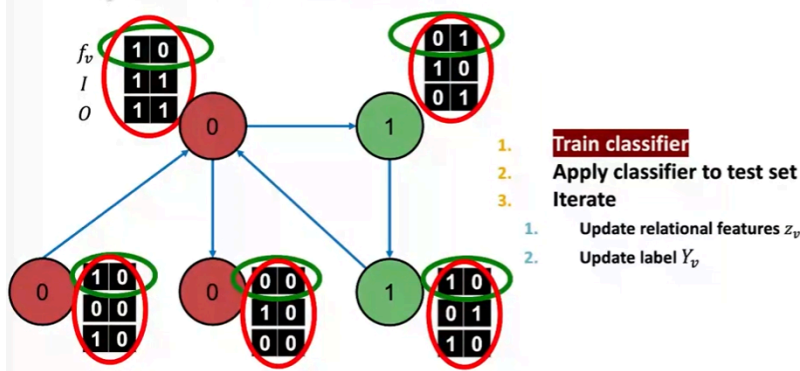


The key insight: you train **two classifiers** on the training set.

- ϕ_1 — uses only the node's own features f_v (ignores neighbors)
- ϕ_2 — uses both the node's features f_v AND the relational features $z_v = [I, O]$ (neighbor info)

The Three Steps (Applied to Test Set)

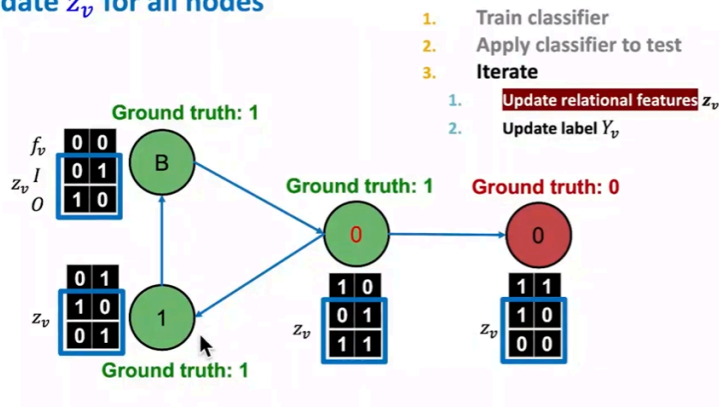
- Step 1: Train
 - On a different **training set**, train two classifiers:
 - Node attribute vector only (green circles): ϕ_1
 - Node attribute and link vectors (red circles): ϕ_2



Train both ϕ_1 and ϕ_2 on a **separate** training graph where all labels are known. This is just standard supervised learning — nothing iterative yet.

- Step 2: Bootstrap with ϕ_1

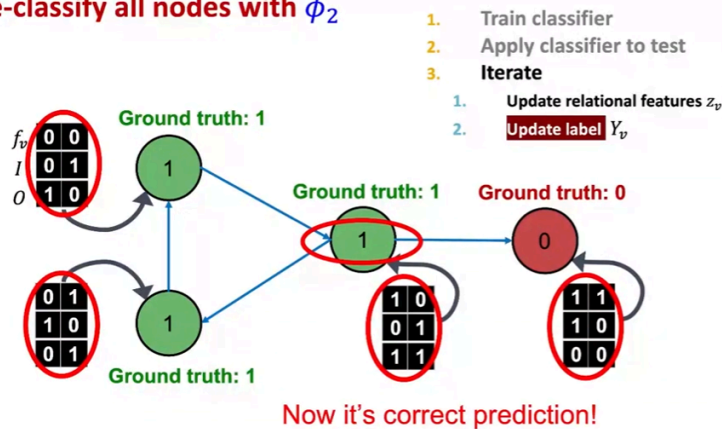
Update z_v for all nodes



Now move to the test set. All labels are ? (unknown). You can't use ϕ_2 yet because you have no neighbor labels to build z_v from! So use ϕ_1 (features-only classifier) to make an **initial guess** for every node's label based purely on f_v .

- Step 3: Iterate (Image 3)

Re-classify all nodes with ϕ_2



3a. Update z_v — For each node, look at its neighbors' *current* predicted labels and rebuild the I and O vectors. (This is what Image 3 shows — the blue boxes highlight the updated z_v values.)

3b. Update Y_v — Now re-run ϕ_2 (the richer classifier) using the freshly updated z_v . This produces new label predictions.

Then repeat 3a → 3b → 3a → 3b... until labels stop changing (convergence) or you hit a max iteration count.